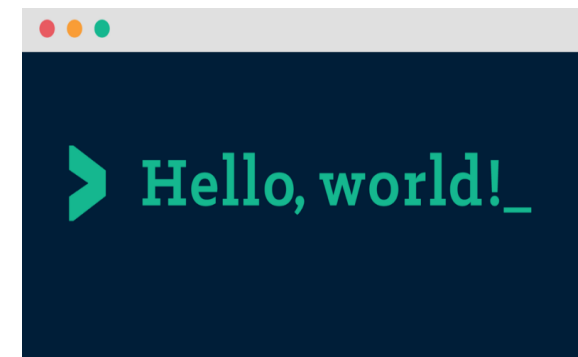
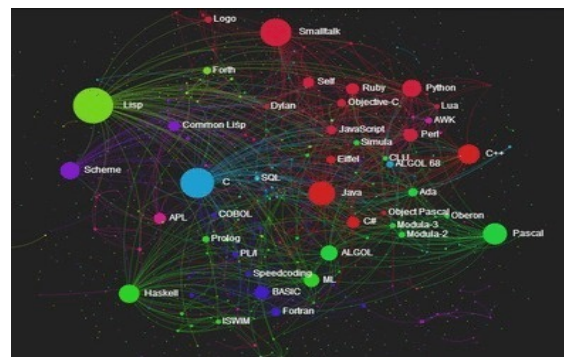


《计算机科学导论》课程

程序设计语言

上海交通大学计算机科学导论课程组



致谢：本课件由范佳豪@CS2018M.SJTU编写，郑继来@CS2019M.SJTU修缮排版；邢磊、张瑞、田作华、潘理提出修订建议，高晓飒终校定稿。



01

程序设计语言概述

02

程序设计语言组成

03

程序设计语言发展

01. 程序设计语言概述：什么是程序设计？(视频来源：GCFGlobal)

01. 程序设计语言概述：什么是程序设计语言？

程序设计语言（又称编程语言）是人与计算机沟通的符号系统，它使用一组记号和规则（语法和语义），让程序员能够准确地定义计算机要使用的数据以及在不同情况下应该采取的行动，从而编写计算机程序来执行特定任务。



程序设计语言填补了
人与计算机交流的鸿沟

$$sum = \sum_{i=1}^n i$$

数学公式

```
while(n>0)
{
    sum = sum + n;
    --n;
}
```

高级编程语言

```
L28  movf    _n,f
      btfsc  STATUS,Z
      goto  L41
      movf    _n,f
      addwf  _sum,f
      btfsc  STATUS,C
      incf   _sum+1,f
      decf   _n,f
      goto   L28
L41
```

符号指令

```
0000100010010011
0001100100000011
0010100000001111
0000100000010011
0000100000010011
0000011110010100
0001100000000011
0000101010010101
0111100000000111
```

计算机指令

本质：不同抽象程度的程序表达

计算思维是蓝图、程序语言是工具

	计算思维	程序语言
角色定位	建筑师 / 系统工程师（设计与统筹）	施工队 / 工匠工具（精确落地）
核心任务	拆解、抽象、模式化，设计可执行的解决方案并评估其可行性	用精确语法和规则，将方案翻译为计算机指令
主要组成	分解；模式识别；抽象；算法设计；验证与迭代；复杂度意识	语法；数据类型；控制流；函数/库与抽象机制（函数、模块、面向对象）；生态与工具链（包管理、调试、测试、CI）
思维层面	想什么、怎么想——从问题空间到解空间	怎么说、怎么做——从解空间到机器指令
与人的关系	提升人类问题解决力，跨学科、跨工具	扩展执行力，与平台和生态相关，强调工程化协作

会一门语言≠具备计算思维； 能跑通≠可扩展与优雅 。**计算思维是气宗，程序语言是剑宗。**
计算思维不限编程课； 同样适用于实验设计、流程优化、数据分析
语言不分“绝对高低级”，要匹配任务与生态：脚本类型快速实现原型，系统语言提升性能/可靠性



01

程序设计语言概述

02

程序设计语言组成

03

程序设计语言发展

02. 程序设计语言组成：语言要素

定义可使用的字符及其码位的集合与映射（如Unicode把“字”映射到码位）。

字符集(Coded Char Set)

ASCII Code Chart																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

词法与记号(Lexical & Token)

<i>alignas</i>	<i>alignof</i>	<i>asm</i>	<i>auto</i>	<i>bool</i>
<i>break</i>	<i>case</i>	<i>catch</i>	<i>char</i>	<i>char16_t</i>
<i>char32_t</i>	<i>class</i>	<i>const</i>	<i>const_cast</i>	<i>constexpr</i>
<i>continue</i>	<i>decltype</i>	<i>default</i>	<i>delete</i>	<i>do</i>
<i>double</i>	<i>dynamic_cast</i>	<i>else</i>	<i>enum</i>	<i>explicit</i>
<i>export</i>	<i>extern</i>	<i>false</i>	<i>float</i>	<i>for</i>
<i>friend</i>	<i>goto</i>	<i>if</i>	<i>inline</i>	<i>int</i>
<i>long</i>	<i>mutable</i>	<i>namespace</i>	<i>new</i>	<i>noexcept</i>
<i>nullptr</i>	<i>operator</i>	<i>private</i>	<i>protected</i>	<i>public</i>
<i>register</i>	<i>reinterpret_cast</i>	<i>return</i>	<i>short</i>	<i>signed</i>
<i>sizeof</i>	<i>static</i>	<i>static_assert</i>	<i>static_cast</i>	<i>struct</i>
<i>switch</i>	<i>template</i>	<i>this</i>	<i>thread_local</i>	<i>throw</i>
<i>true</i>	<i>try</i>	<i>typedef</i>	<i>typeid</i>	<i>typename</i>
<i>union</i>	<i>unsigned</i>	<i>using</i>	<i>virtual</i>	<i>void</i>
<i>volatile</i>	<i>wchar_t</i>	<i>while</i>		

词法规则把字符流切分并归类为记号；记号是词法输出的最小语法单位（如标识符、关键字、字面量、运算符）。

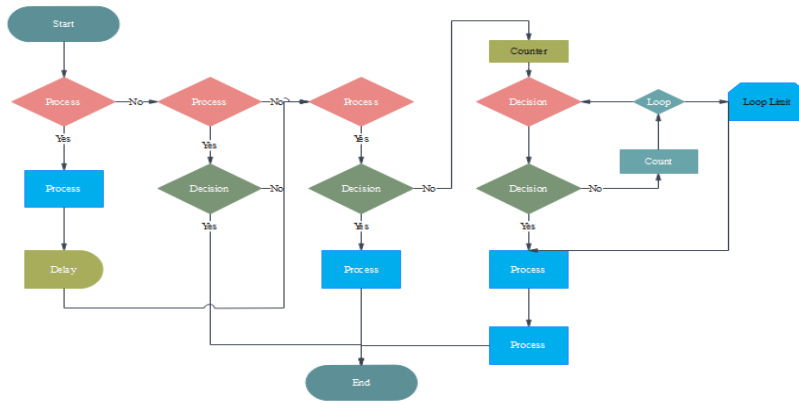
规定记号如何组合成更大结构（表达式、语句、声明等）的形式规则。

语法 (Syntax)

```
public void processData() {
    do {
        int data = getData();

        if (data < 0)
            performOperation1(data);
        else
            performOperation2(data);
    } while (hasMoreData());
}
```

语义 (Semantics)



为语法结构赋予含义的规则，包括静态语义（类型、作用域、名称解析等）与动态语义（执行时的行为与结果）。

02. 程序设计语言组成：四大基本成分

数据成分：用以描述程序所涉及的对象——数据

- $A = 3 + 2$

运算成分：用以描述程序中所包含的运算（算术运算、逻辑运算、字符串运算等）

- $A = 3 + 2$

控制成分：用以控制程序中所含语句的执行顺序

- `if (P) S1 else S2`
- `while (P) do S1`

传输成分：用以描述程序中的数据传输操作

- 赋值 (=)
- 输入/输出
- `scanf(...)`
- `printf(...)`
- `read(...)`
- `write(...)`

02. 程序设计语言组成：数据成分

数据是客观事物在计算机内的（格式化）表示，是程序所操作和处理的对象

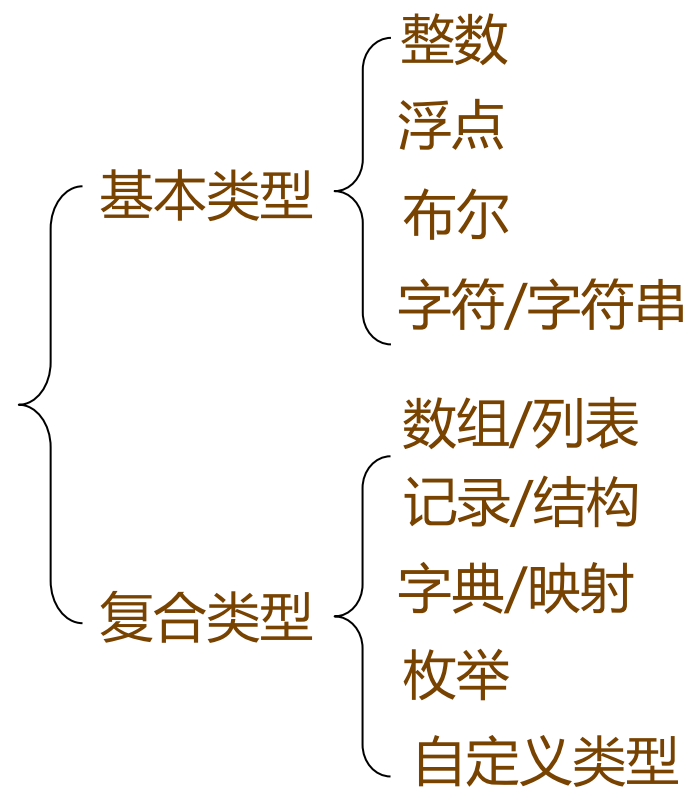
数据=值 (value) +类型 (type) +存储 (memory)

在多数静态类型语言中要**先声明后使用**；

在动态类型语言中可不显式声明类型，但运行时仍有类型。

数据说明的内容包括：

- 变量还是常量
- 它们的标识符（符号表示）
- 它们的数据类型
- 它们的初始值
-



类型与错误：何时检查、何时出错

静态类型 (compile time)

含义：大部分类型错误在编译期被发现

例：`int x = "hi";` // 编译不通过 (Java/C/C#/Rust)

动态类型 (run time)

含义：变量可绑定任意类型的值，类型错误多在运行期暴露

例：`x = "hi"; print(x + 1)` // 到此行才可能报错 (Python/JS)

强类型(strong):

禁止或严格限制隐式类型转换，类型不匹配即报错。

`"2" + 1` → 错；需 `int("2") + 1` (Python)

弱类型(weak): 允许较多隐式转换，可能“能跑但不一定对”。

`"2" + 1` → `"21"`, `"2" - 1` → `1` (JavaScript)

错误类型与出现时机

- 语法错误：漏标点 → 编辑/解析阶段
- 静态语义/类型错误：类型不匹配、未声明变量 → 编译/静态检查
- 运行时错误：除零、空指针、下标越界、类型断言失败 → 执行时
- 逻辑错误：程序能跑但结果不对 → 测试/使用中

类型系统是“提前发现问题的自动化工具”，影响“什么时候报错、谁来兜底（编译器还是运行时）、以及需不需要显式转换”。

02. 程序设计语言组成：运算成分

运算符

- 算术运算符：+ - * / % ++ --
- 关系运算符：< <= == > >= !=
- 逻辑运算符：&& || !
- 位运算符：<< >> & | ~ ^
- 赋值运算符：= 及其扩展
- 条件运算符：? :
- 逗号运算符：,
- 指针运算符：* &
- 分量运算符：. ->
- 下标运算符：[]
- 其它：()

表达式由变量、常量、函数、运算符和括号组成，它用来计算一个值。

例如，

$$\sin(45*3.14/180) \\ +(exp(10)+log(10))/sqrt(x+y+1)$$

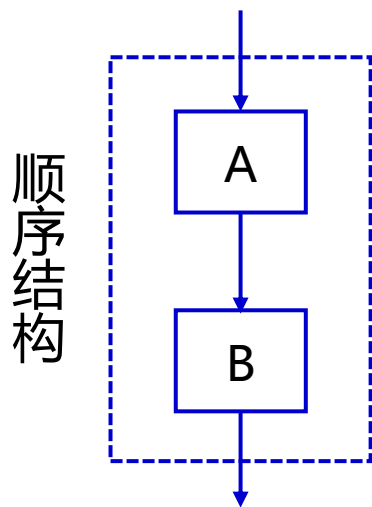
用来计算下面公式的值：

$$\sin 45^{\circ} + \frac{e^{10} + \ln 10}{\sqrt{x + y + 1}}$$

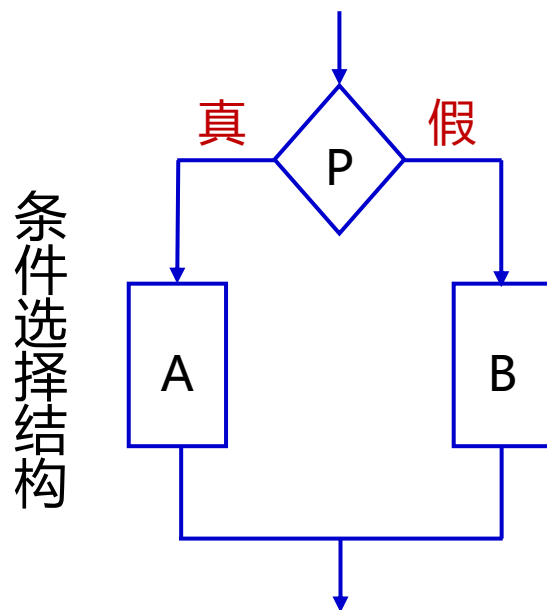
02. 程序设计语言组成：控制成分

控制成分用来提供一种**基本框架**，在此基本框架的支持下，将数据和对数据的运算组合成程序

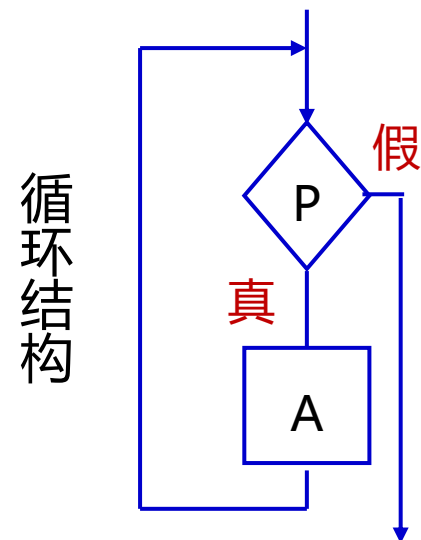
三种基本控制结构：



由一组顺序执行的计算操作（或语句）组成



由条件（P）和两个供选择的操作A和B组成



由条件（P）和一个供重复执行的操作A组成

02. 程序设计语言组成：C语言实现两数求和

C: 静态、强类型、编译

```
/* calculate the sum of a and b */
```

```
#include <stdio.h>
```

预处理命令

```
/* This is the main program */
```

注释

```
int main(void){
```

```
    int a, b, sum;
```

数据类型说明

```
    a = 10;
```

赋值语句

```
    b = 24;
```

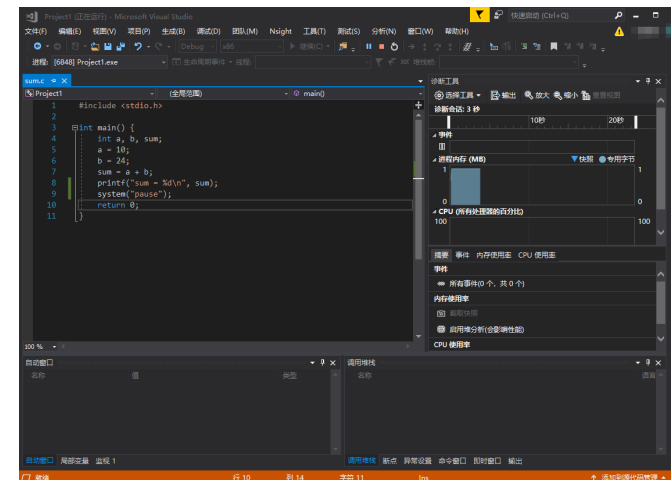
```
    sum = a + b;
```

运算语句

```
    printf("sum = %d\n", sum);
```

输出语句

```
}
```



操作界面 (Visual Studio)



输出效果

02. 程序设计语言组成：Python语言实现两数求和

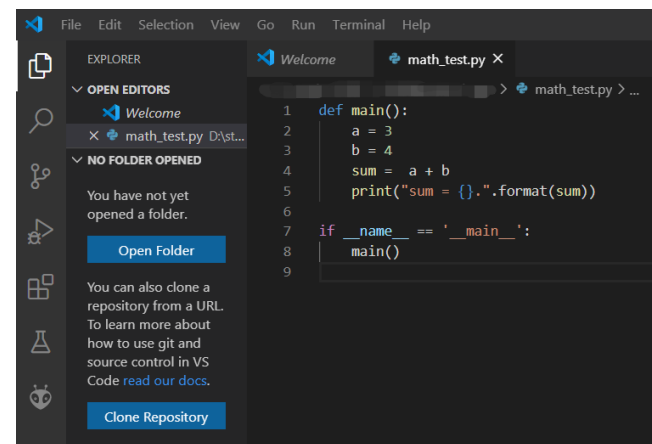
Python: 动态、强类型、解释执行

```
# This is the main program ← 注释  
  
def main(): ← 函数定义  
    a = 3 ← 变量定义与赋值  
    b = 4 ← 变量定义与赋值  
    sum = a + b ← 运算语句  
    print("sum = {}".format(sum)) ← 输出语句
```

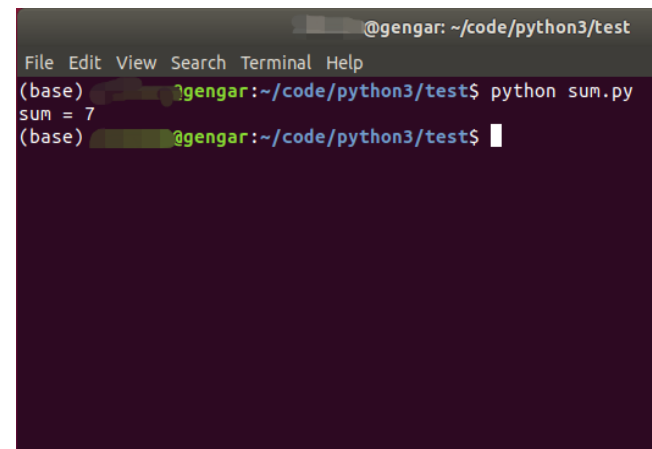
```
# calculate the sum of a and b ← 注释  
if __name__ == '__main__': ← 函数调用  
    main()
```

这是Python编程的最佳实践之一，让代码既可以作为独立程序运行，也可以作为模块被其他程序导入使用。

关于__name__与__main__: 知乎 <https://www.zhihu.com/question/49136398>



操作界面 (VS Code)



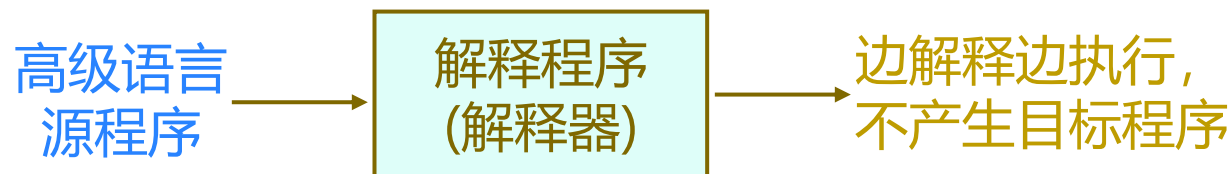
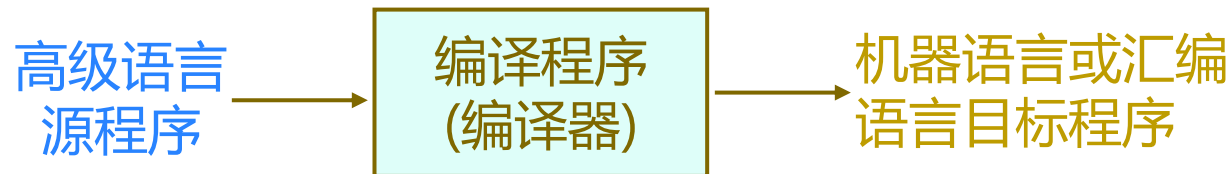
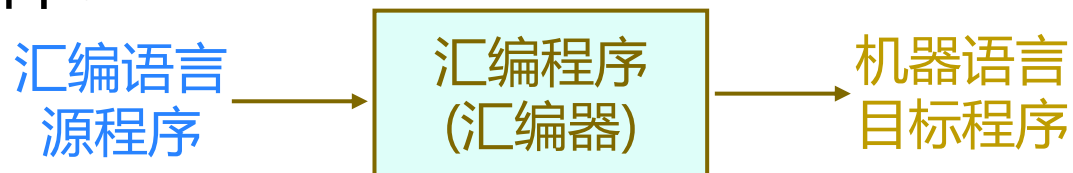
输出结果

02. 程序设计语言组成：翻译程序

翻译程序：把甲语言编写的程序翻译为等价的乙语言编写的程序。
其中甲语言程序称为源程序，乙语言程序称为目标程序



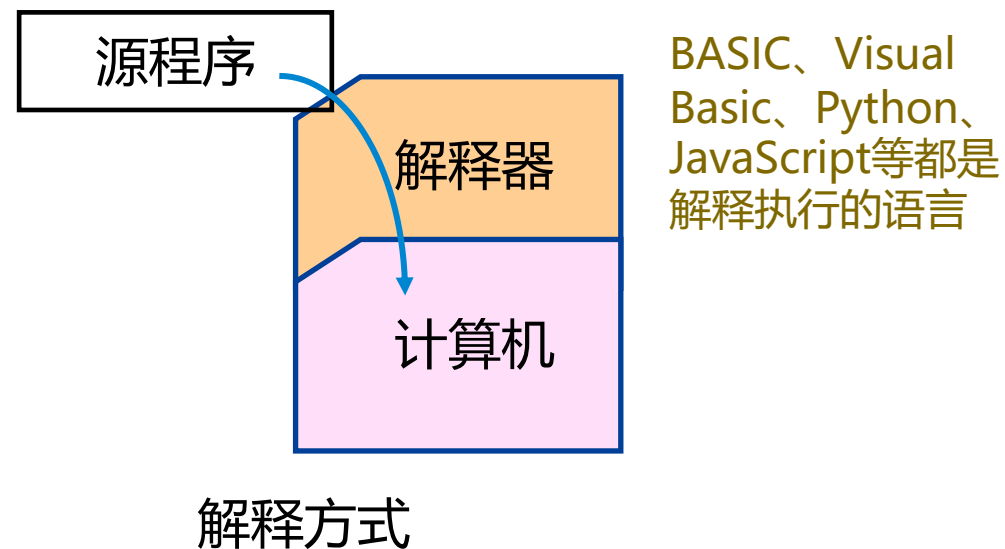
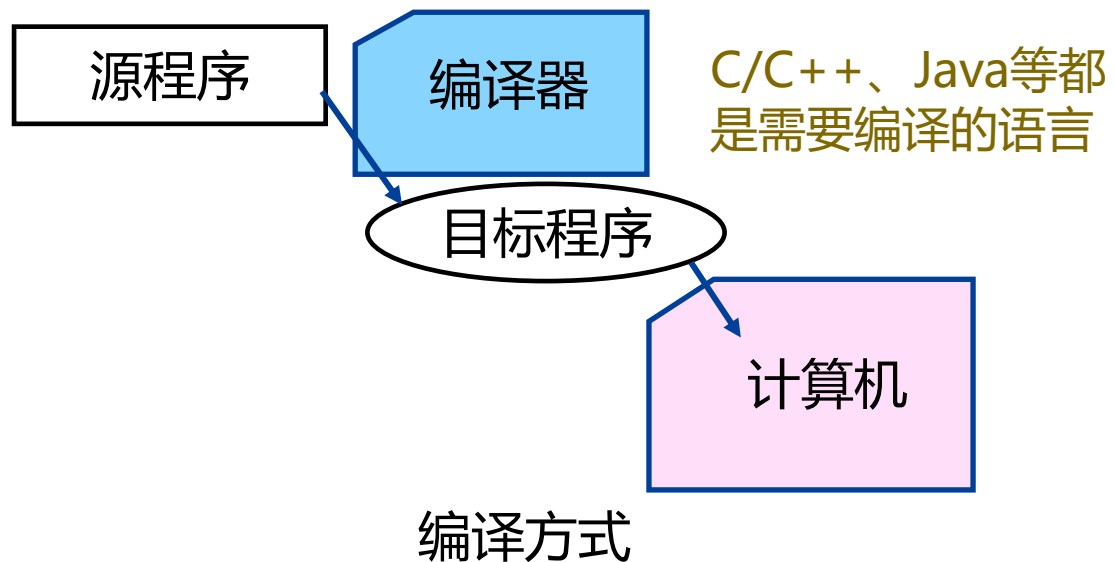
常用的有下列几种：



02. 程序设计语言组成：编译程序与解释程序的区别

编译程序：把源程序编译为机器语言目标程序后，再由计算机运行
(相当于“笔译”)

解释程序：解释器直接解释并且执行源语言程序，不产生目标程序
(相当于“口译”)



02. 程序设计语言组成：编译器与解释器

C/C++ 编译器

- GCC (GNU Compiler Collection)
- LLVM+Clang
- Watcom C/C++
- Digital Mars
- Microsoft Visual C/C++
- Intel C++
- TCC (Tiny C Compiler)

Python解释器

- CPython
- IPython
- PyPy
- Jython
- IronPython

字节码 / 虚拟机 / JIT 与 AOT

流程：源代码 → 编译器 → 字节码 (.class/.dll/.pyc) → 虚拟机 (JVM/CLR/Python VM)

→ 运行方式：解释 | JIT (热点编译) | AOT (预编成本地码)

核心概念

字节码 (Bytecode): 跨平台中间代码，便于移植优化

虚拟机 (VM): 抽象计算机，统一运行环境

JIT (Just-In-Time): 运行时将热点代码转为本地码

AOT (Ahead-Of-Time): 部署前编译为本地机器码

典型生态

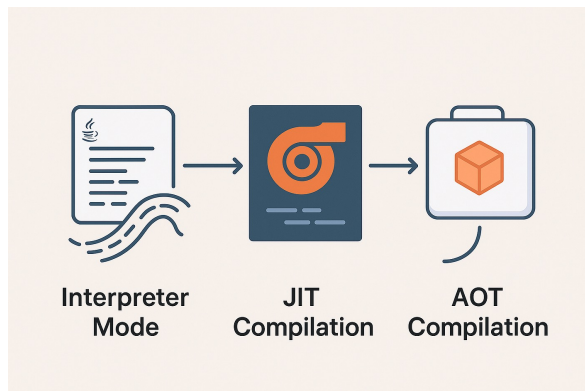
JVM 生态: Java, Kotlin → .class → JVM (解释+JIT)

.NET 生态: C#, F# → IL → CLR (JIT)

Python: .py → .pyc → CPython VM (解释) | PyPy (JIT)

JavaScript: 源码 → V8/SpiderMonkey 引擎 (内置字节码+JIT)

Swift: → 本地机器码 → 直接运行 (AOT)



场景与权衡

追求跨平台 → 用字节码 + VM。

追求启动快 → 选 AOT 或混合模式。

追求峰值性能 → JIT 运行期优化是王牌，服务器应用。

设备资源受限 → AOT 更轻量，移动应用。

需要安全/管理 → VM 提供强大沙箱与工具。

编译/解释谈“从源到可执行”；字节码/VM/JIT/AOT谈“运行期怎样执行与优化”。



01

程序设计语言概述

02

程序设计语言组成

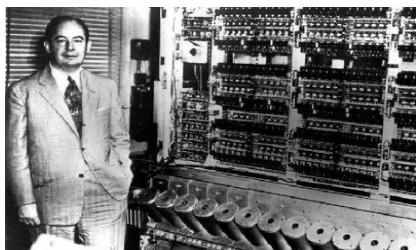
03

程序设计语言发展

03. 程序设计语言发展：里程碑

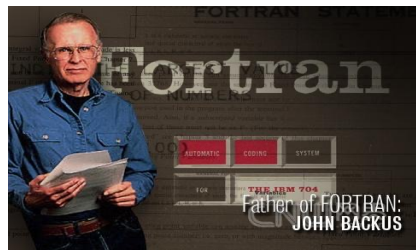
1946 机器语言

ENIAC诞生，J von Neumann 存储程序架构奠定现代计算机与软件的前提，程序与数据同存。



1957 高级语言

IBM的John Backus小组发布FORTRAN，第一个被广泛使用的高级语言，主要用于科学计算。



1967 面向对象

Ole-Johan Dahl, Kristen Nygaard 发明Simula 67，定义类、对象和继承，推动OO范式从理论到实践。



1990s Web与虚拟机

James Gosling跨平台虚拟机（Java）、Brendan Eich浏览器脚本（JavaScript）和Guido van Rossum 简洁脚本化（Python），奠定Web和现代生态基础。



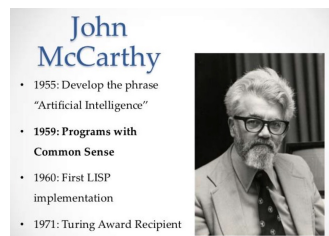
1951 汇编语言

Grace Hopper团队发明了第一个编译器，成为高级语言的先驱。



1958 函数式编程

John McCarthy设计LISP语言，开创函数式编程范式，引入垃圾回收和符号计算，对AI和现代语言影响深远。



1972 现代编程

Dennis Ritchie在贝尔实验室创造C语言，兼具高效与灵活



2010s 安全化

Graydon Hoare等Mozilla团队发明Rust，强调内存安全和所有权模型，平衡性能与可靠性，影响系统级编程未来。



03. 程序设计语言发展：机器语言

机器语言：计算机的指令系统，使用二进制编码表示
用机器语言编程序，就是直接使用二进制代码编写程序
为了简化表达，通常也使用十六进制代替二进制记录指令（如图）

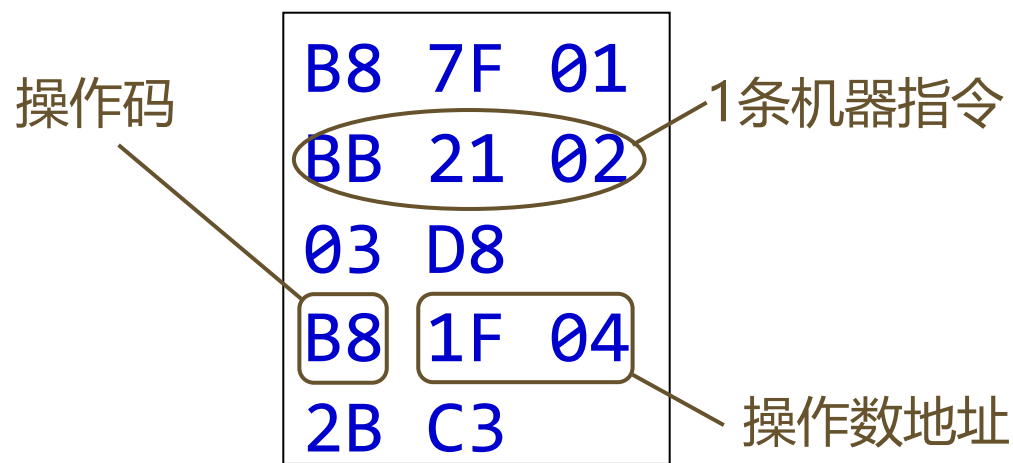
优点：

- 可以直接被计算机执行

缺点：

- 记不住、难理解、效率低、不易维护
- 不同的机器语言程序，相互不兼容

现在已不直接用机器语言编程序！



计算 $1055 - (383 + 545)$ 的5条机器指令

03. 程序设计语言发展：汇编语言

汇编语言用助记符号来表示机器指令中的操作符与操作数

优点：

- 操作数直接使用十进制
- 程序相对容易理解

缺点：

- 大型程序难以开发
- 依赖于具体计算机

机器语言程序

B8	7F	01
BB	21	02
03	D8	
B8	1F	04
2B	C3	

计算 $1055 - (383 + 545)$ 的5条机器指令

对应的汇编语言程序

汇编语言程序

MOV	AX	383
MOV	BX	545
ADD	BX	AX
MOV	AX	1055
SUB	AX	BX

将383传送到AX寄存器

将545传送到BX寄存器

将BX内容加AX内容，结果在BX中

将1055传送到AX寄存器

将AX内容减BX内容，结果在AX中

03. 程序设计语言发展：高级语言

高级语言是人们为了克服汇编语言的缺陷，提高编程和维护的效率而设计的

特点：

- 接近人们日常使用的自然语言（主要是英语），容易理解、记忆和使用
- 可以在不同计算机上通用
- 对使用的符号、词汇、语法和语义等各种语言成分都有严格的规定

意义：降低了程序设计的难度，使计算机的发展进入新的阶段

高级语言是目前程序设计语言的主流！

汇编语言程序

MOV	AX	383
MOV	BX	545
ADD	BX	AX
MOV	AX	1055
SUB	AX	BX

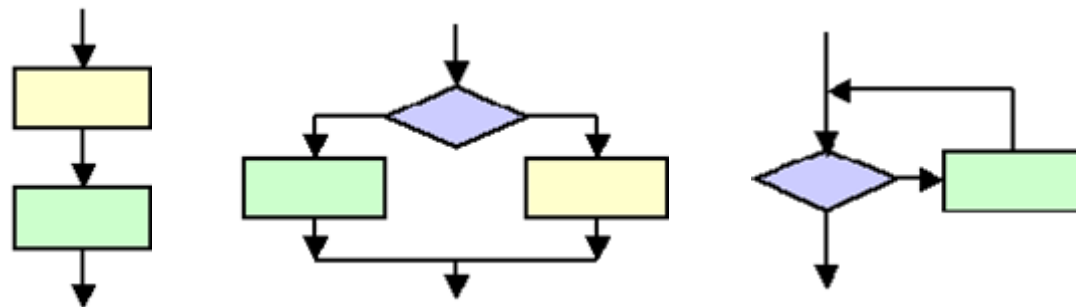


$S = 1055 - (383 + 545)$

03. 程序设计语言发展：高级语言编程特性

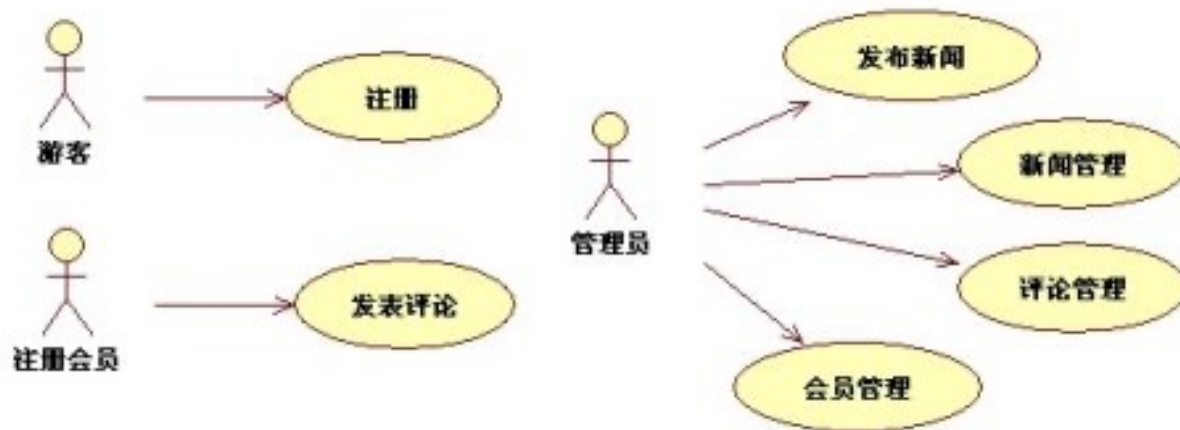
结构化

- 使用子程序、区块、分支、循环等结构，取代跳转语句（goto）
- 改善程序的品质与可读性，避免面条式代码，减少开发时间



面向对象

- 对现实事物特征及变化规律进行抽象，包括与事物关联的数据和对数据的操作
- 事物概念被建模为类（model），事物实例被封装为对象（object）
- 对象交互与现实场景对应，帮助把握问题本质，提高程序设计效率



03. 程序设计语言发展：高级语言编程特性

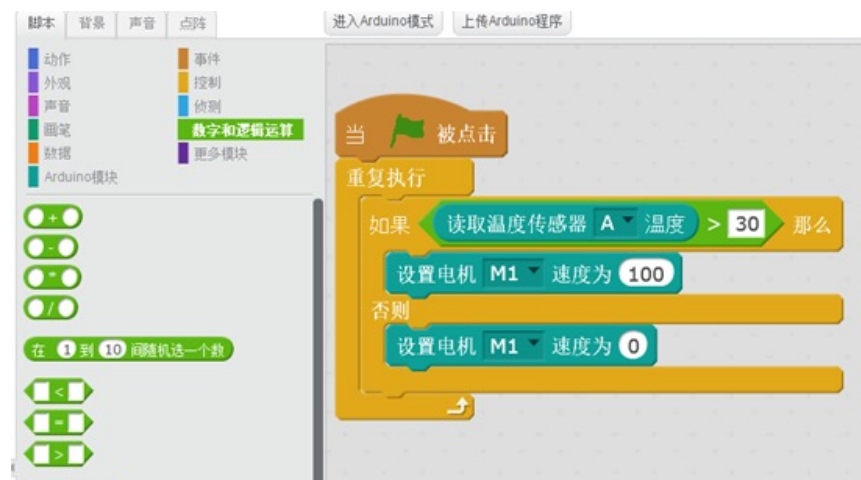
图形用户界面 (GUI)

- 用图形展示与计算机交互的界面
- 与通过键盘输入文本命令来完成例行任务的字符界面相比，提供图形用户界面的程序更加直观明了



图形化编程

- 去除枯燥的编程语法，使用图形化和模块化的编程方式进行程序创造
- 学习成本低，适合简单程序的开发，培养编程思维



03. 程序设计语言发展：编程范式（分类法）

编程范式是一种基本的编程方法或风格，它为设计和实现计算机程序提供了一套原则、概念和技术。它决定了程序员应该如何思考和构建代码，选择正确的范式可以极大地影响程序的效率、可维护性和可扩展性。

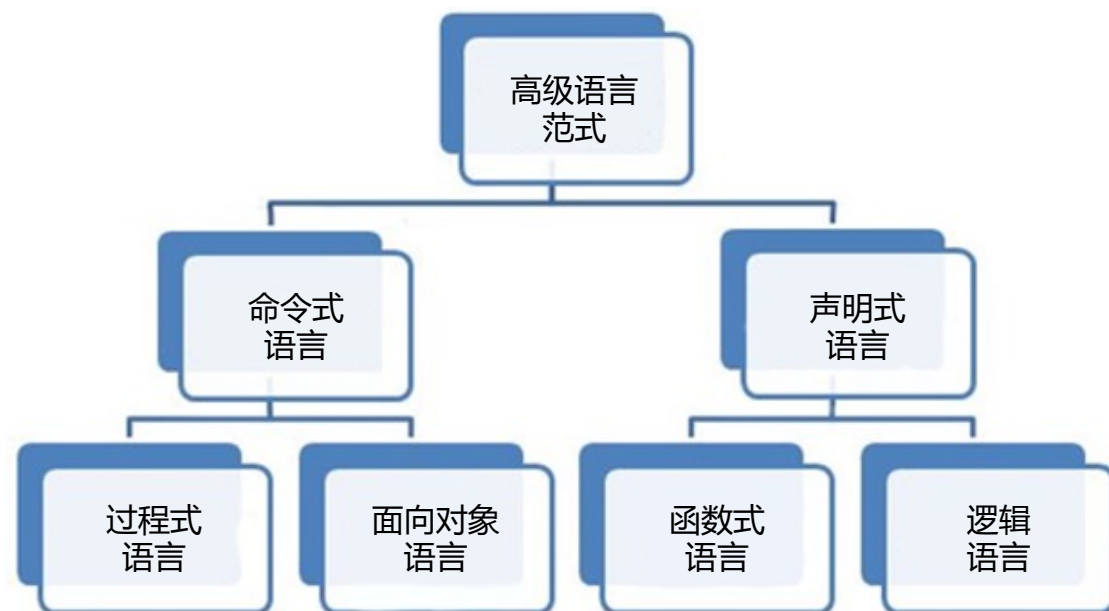
命令式/过程式 (Imperative/Procedural): 程序围绕作数据的过程或函数构建。Pascal、C、Go、Rust

面向对象 (Object-Oriented): 围绕对象的概念展开，对象是类的实例。它将代码组织成封装数据和行为的对象。Java、C#、Kotlin、Swift、Python

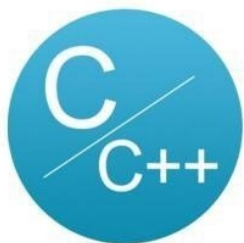
函数式 (Functional): 将计算视为对数学函数的评估，并避免改变状态和可变数据。Haskell、F#; Scala、Kotlin、JavaScript

声明式(Declarative): 侧重于描述所需的结果，而不是指定实现它的详细步骤。SQL

逻辑语言 (Logic): 数理逻辑约束的列举，描述事实与规则，而非具体解决过程。Prolog

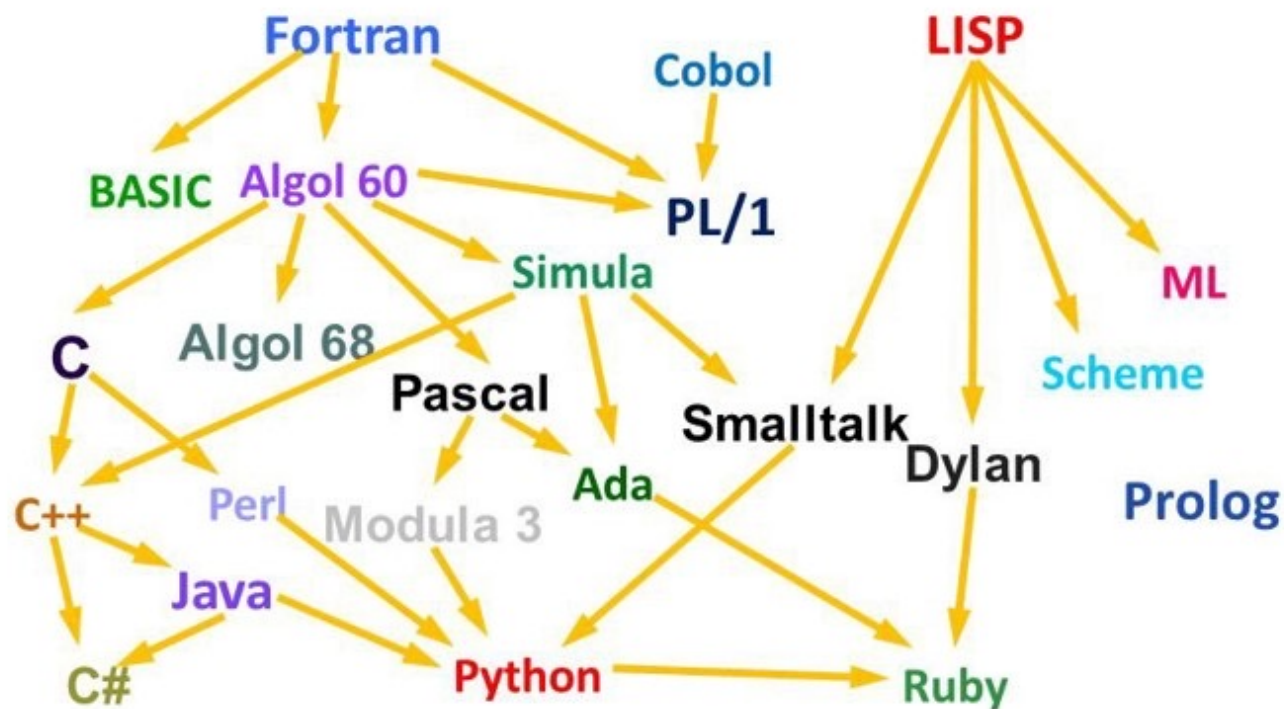


03. 程序设计语言发展：高级语言演进关系



A family tree of languages

Some of the 2400 + programming languages



*图片来源于网络



03. 程序设计语言发展：PYPL排名(Popularity of Programming Language)

Worldwide, Sept 2025 :				
Rank	Change	Language	Share	1-year trend
1		Python	29.69 %	+0.2 %
2		Java	14.72 %	-0.7 %
3	↑	C/C++	9.27 %	+2.5 %
4	↓	JavaScript	6.79 %	-1.4 %
5	↑	R	5.26 %	+0.6 %
6	↓	C#	4.81 %	-1.8 %
7	↑↑↑↑	Objective-C	4.17 %	+1.7 %
8	↓	PHP	3.34 %	-0.8 %
9	↑	Rust	2.73 %	+0.2 %
10	↓	Swift	2.72 %	+0.0 %
11	↓↓↓	TypeScript	2.45 %	-0.5 %
12	↑↑↑	Ada	2.11 %	+1.1 %
13	↓	Go	1.56 %	-0.6 %
14		Matlab	1.49 %	+0.0 %
15	↓↓	Kotlin	1.37 %	-0.6 %

16	↑	Powershell	1.1 %	+0.1 %
17	↑↑	Ruby	1.05 %	+0.1 %
18	↑↑	Lua	0.97 %	+0.2 %
19	↓↓↓	VBA	0.95 %	-0.1 %
20	↓↓	Dart	0.88 %	-0.1 %
21	↑	Abap	0.6 %	+0.0 %
22	↑↑	Julia	0.46 %	+0.1 %
23	↓↓	Scala	0.46 %	-0.1 %
24	↓	Visual Basic	0.29 %	-0.2 %
25	↑	Zig	0.19 %	-0.0 %
26	↑	Cobol	0.15 %	-0.0 %
27	↑	Perl	0.13 %	+0.0 %
28	↑↑	Delphi/Pascal	0.13 %	+0.1 %
29	↓↓↓↓	Groovy	0.11 %	-0.1 %
30	↓	Haskell	0.06 %	+0.0 %

© Pierre Carbonnelle, 2023



- 根据google搜索不同语言教程次数进行排名
- 2025年9月版
- 对比与1年前的排名差距
- © Pierre Carbonnelle

标准	要点	优势示例	挑战示例
生态	语言的库、框架与社区支持	Python（丰富库）、 JavaScript（Web生态）	Go（生态较新）
工具	IDE、调试器、构建工具的可用性	Java（IntelliJ）、 C#（Visual Studio）	Rust（工具链复杂）
学习曲线	语言语法与上手难度的友好度	Python（简洁）、 JavaScript（易上手）	C++（语法复杂）、 Rust（严格规则）
性能	运行速度、资源占用与优化潜力	C/C++（接近硬件）、 Go（AOT高效）	Python（解释执行较慢）
安全	内存安全、类型检查与漏洞防护	Rust（内存安全）、 Java（沙箱）	C（易出内存错误）

程序设计语言概述

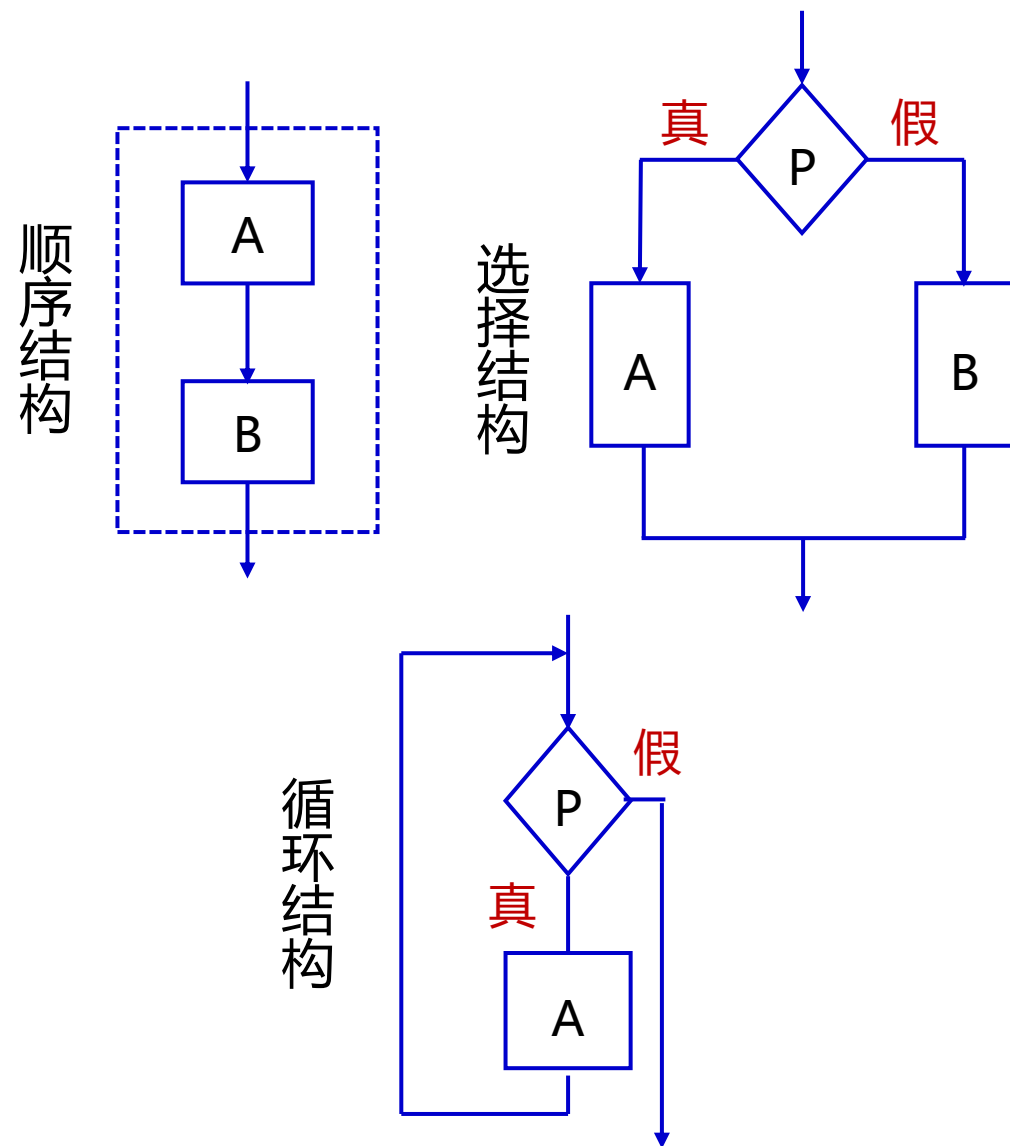
- 什么是程序设计
- 什么是程序设计语言

程序设计语言组成

- 语言要素（字符集、词汇、语法、语义）
- 四大基本成分（数据、运算、控制、传输）
- 示例：Python语言实现两数求和
- 编译器：程序编译基本规则及介绍

程序设计语言发展

- 里程碑（机器语言、汇编语言、高级语言...）
- 高级语言编程特性、范式（分类法）
- 高级语言演进关系、新动态



谢谢!

